

SIKSHA 'O' ANUSANDHAN
(Deemed To Be University)
Accredited (3rd Cycle) by NAAC with A⁺⁺ Grade

Admission Batch:

Session:

Assignment of
MACHINE LEARNING CONCEPT -1 (CSE 3967)

Submitted by

Name: _____

Registration No.:_____Class Serial No.:_____

Section:_____Assignment No.:_____

Semester:_____Branch:_____



Department of Computer Science & Engineering
Faculty of Engineering & Technology (ITER)
Jagamohan Nagar, Jagamara, Bhubaneswar, Odisha - 751030

Assignment – 1 (Chapter1-2)

Part A – Theory Questions

Q1.

- (a) From the dataset of a hospital (age, blood pressure, disease type, treatment success: Yes/No), mark which features are numerical and which are categorical.
 - (b) Why might using a categorical column (like disease type) as a number cause wrong conclusions?
 - (c) Which ML algorithm would naturally handle categorical data well? Which one fits numerical better?
-

Q2.

- (a) A company records the salaries of employees. If one CEO earns 1 crore while most others earn below 50,000, would you use mean or median to describe central tendency? Why?
 - (b) Define variance in your own words with an example.
 - (c) How does standard deviation give more intuition about data spread compared to variance?
-

Q3.

- (a) A student memorizes last year's question paper and scores well in mock tests but fails in the real exam. How is this similar to ML overfitting?
 - (b) List two strategies to improve generalization in models.
 - (c) Explain with an analogy why underfitting is as harmful as overfitting.
-

Q4.

- (a) A survey has missing "income" data for 10% of participants. Suggest two ways to address this issue.
 - (b) In a dataset of house prices, one record shows a price $100\times$ higher than others. What is this called? How can it affect predictions?
 - (c) Which is worse for a model — systematic missing values or random missing values? Explain briefly.
-

Q5.

- (a) A histogram of exam scores shows a long left tail. What does it suggest about student performance?
 - (b) A boxplot shows several dots above the whisker — what does that mean?
 - (c) Why might scatter plots help you guess relationships before fitting a model?
-

Q6.

- (a) In predicting house prices, which would be the features and which is the target?
 - (b) If one feature is strongly correlated with the target, how does it help the model?
 - (c) Can we ever have multiple targets? Give an example.
-

Q7.

- (a) Classify these scenarios: grouping songs by similarity, predicting next month's rainfall, teaching a robot to balance.
 - (b) Why is supervised learning more common in industry?
 - (c) Give one benefit of unsupervised learning despite being harder to evaluate.
-

Q8.

- (a) A recruitment dataset contains more male than female applicants. What type of data bias is this?
 - (b) How can such bias affect ML model predictions?
 - (c) Suggest one way to reduce the impact of data bias.
-

Q9.

- (a) If you have only 50 data points, would you use a very complex model or a simple one? Why?
- (b) What danger arises if the model has too many parameters compared to the dataset size?
- (c) Why do simpler models often generalize better on small datasets?

Part B – Lab Questions

Q1.

- (a) Create a 1D NumPy array with values 1 to 20.
- (b) Extract all prime numbers from it.
- (c) Compute the mean and variance of the extracted primes.

Q2.

- (a) Create a 4×4 NumPy array with numbers 1 to 16.
- (b) Extract the 2×2 bottom-left sub-matrix.
- (c) Compute the determinant of the sub-matrix.

Q3.

Create a DataFrame with 5 students and marks in 3 subjects.

- (b) Add a column for total and average marks.
- (c) Identify the topper and print their name with average.

Q4.

- (a) Simulate 1000 coin tosses using NumPy (1=Head, 0=Tail).
- (b) Count frequency of heads and tails.
- (c) Estimate probability of heads. Is it close to 0.5? Why/Why not?

Q5.

- (a) Create a DataFrame of employees with columns: ID, Name, Salary.
- (b) Add a Bonus column = 10% of Salary.
- (c) Display employees with salary above average.

Q6.

- (a) Create a 3×3 NumPy array with values 1 to 9.
- (b) Find its transpose and inverse.
- (c) Verify that $A \times A^{-1} \approx I$.

Q7.

- (a) Generate random daily temperatures (30 values, range 20–40°C).
- (b) Find the hottest and coldest day.
- (c) Compute mean, median, and standard deviation of temperatures.

Q8.

- (a) Create a Pandas Series of marks for 8 students.
- (b) Replace all marks below 40 with “Fail”.
- (c) Count how many students passed.

Q9.

- (a) Generate 500 random integers between 1 and 6 (simulate dice rolls).
- (b) Count how many times each face appears.
- (c) Compute relative frequencies and compare with theoretical $1/6$.

Q10.

- (a) Create a DataFrame with 6 products (Name, Quantity, Price).
- (b) Add a column “Total = Quantity $\times$ Price”.
- (c) Find which product generated maximum sales revenue.

Q11.

- (a) Create a DataFrame with some missing values (NaN).
- (b) Fill missing values with column mean.
- (c) Drop rows where more than 1 value is missing.

Q12.

- (a) Create a NumPy array with integers from 1 to 30.
- (b) Reshape it into a 5×6 matrix.
- (c) Extract all even numbers from the matrix and compute their average.

Q13.

- (a) Create a DataFrame of 6 students with columns: Name, Age, Marks.
- (b) Select only students who scored above the overall average marks.
- (c) Display names of students younger than 20 whose marks are above 60.

Q14.

- (a) Create a Pandas DataFrame with 5 employees having columns: Name, Tasks_Completed, Hours_Worked.
- (b) Add a new column Efficiency = Tasks_Completed / Hours_Worked.
- (c) Identify the employee with the **highest efficiency** and print their name and value.

Part C – Case Study

Case Study 1 – Student Performance Analytics

You are given data of 100 students containing their marks in **Math, Science, and English**.

- (a) Using Pandas, calculate the average marks for each student and identify the top 5 performers.
 - (b) Compute subject-wise average and standard deviation; comment which subject shows the highest variation.
 - (c) Draw a boxplot for each subject and comment on outliers.
-

Case Study 2 – Sales Data Exploration

A company records sales transactions in a dataset with columns: **Product, Quantity, Price, Region**.

- (a) Add a new column $\text{Total} = \text{Quantity} \times \text{Price}$ and compute overall revenue.
 - (b) Group data by region and identify which region contributes the most sales.
 - (c) Plot a histogram of total sales per product and discuss which products are underperforming.
-

Case Study 3 – Predicting Fuel Efficiency (Auto MPG Dataset)

You are analyzing the **Auto MPG dataset (UCI Repository)** with features: mpg, cylinders, displacement, horsepower, weight, acceleration, model year, origin.

- (a) Compute the correlation of mpg with each numeric feature and discuss which factors influence fuel efficiency most.
- (b) Plot a scatter plot of weight vs mpg; describe the trend you observe.

ANSWERS

Part A - Theory

Q1

- (a) Numerical: Age, Blood Pressure. Categorical: Disease Type, Treatment Success.
- (b) Treating categorical values as numbers may create false numerical relationships.
- (c) Categorical - Decision Tree; Numerical - Linear Regression.

Q2

- (a) Median - less affected by extreme salary (CEO outlier)
- (b) Variance measures how far data points deviate from the mean. e.g., marks spread around average.
- (c) Standard deviation is in the same unit as data, showing actual spread intuitively.

Q3

- (a) Like overfitting - model memorizes training data but fails on new data.
- (b) Use cross-validation and regularization (or collect more data).
- (c) Underfitting is like not learning enough - model too simple to capture patterns.

Q4

- (a) Fill missing values with mean/median or remove rows.
- (b) It's an outlier; can skew model and predictions.
- (c) Systematic missing values are worse - they bias the model.

Q5

- (a) Left tail → most students scored high, few low scores.
- (b) Dots above whisker = outliers (high values)
- (c) Scatter plots show relationships and correlations visually before modeling.

Q6

- (a) Features = size, location etc.; Target = price.
- (b) Strong correlation helps predict target accurately.
- (c) Yes - e.g., predicting temperature and humidity together.

Q7

- (a) Grouping songs - Unsupervised; Rainfall prediction - Supervised; Robot balancing - Reinforcement.
- (b) Labeled data is readily available and results are easier to evaluate.
- (c) It can discover hidden patterns without labels.

Q8

- (a) Sampling bias (or gender bias)
- (b) Model may favour male applicants in predictions.
- (c) Balance dataset or use bias-mitigation techniques.

Q9

- (a) Use a simple model - complex models overfit small data.
- (b) Too many parameters cause overfitting and poor generalization.
- (c) Simpler models learn core patterns and avoid noise fitting.



Q1

```
In [4]: import numpy as np
# (a) Create 1D array with values 1 to 20
arr = np.arange(1, 21)
# (b) Function to check prime
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n**0.5)+1):
        if n % i == 0:
            return False
    return True
# Extract prime numbers
primes = np.array([x for x in arr if is_prime(x)])
print("Primes:", primes)
# (c) Compute mean and variance
mean_primes = primes.mean()
var_primes = primes.var()
print("Mean of primes:", mean_primes)
print("Variance of primes:", var_primes)
```

```
[ 2  3  5  7 11 13 17 19]
Mean of primes 9.625
Variance of primes 35.734375
```

Q2

```
In [21]: import numpy as np
# (a) Create 4x4 array with numbers 1 to 16
A = np.arange(1, 17).reshape(4, 4)
print("4x4 Array:\n", A)
# (b) Extract 2x2 bottom-left sub-matrix
# Bottom-left elements: rows 2-3 (index 2,3), columns 0-1 (index 0,1)
sub_matrix = A[2:, 0:2]
print("\n2x2 Bottom-Left Sub-matrix:\n", sub_matrix)
# (c) Compute determinant of the sub-matrix
det = np.linalg.det(sub_matrix)
print("\nDeterminant of Sub-matrix:", det)
```

```
4x4 Array:
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]
 [13 14 15 16]]
```

```
2x2 Bottom-Left Sub-matrix:
[[ 9 10]
 [13 14]]
```

```
Determinant of Sub-matrix: -4.0000000000000009
```


Q3

```
In [22]: import pandas as pd
# (a) Create DataFrame
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eva'],
    'Math': [85, 78, 92, 88, 76],
    'Science': [90, 82, 85, 91, 80],
    'English': [78, 88, 80, 85, 89]
}
df = pd.DataFrame(data)
# (b) Add columns for Total and Average
df['Total'] = df[['Math', 'Science', 'English']].sum(axis=1)
df['Average'] = df[['Math', 'Science', 'English']].mean(axis=1)
# (c) Identify topper
topper = df.loc[df['Average'].idxmax()]
print("Student Data with Total and Average:\n", df)
print("\nTopper:")
print("Name:", topper['Name'])
print("Average Marks:", topper['Average'])
```

Student Data with Total and Average:

	Name	Math	Science	English	Total	Average
0	Alice	85	90	78	253	84.333333
1	Bob	78	82	88	248	82.666667
2	Charlie	92	85	80	257	85.666667
3	David	88	91	85	264	88.000000
4	Eva	76	80	89	245	81.666667

Topper:

Name: David

Average Marks: 88.0

Q4

```
In [4]: import numpy as np
# (a) Simulate 1000 coin tosses
# 1 represents Head, 0 represents Tail
tosses = np.random.randint(0, 2, 1000)
# (b) Count frequency of heads and tails
heads_count = np.sum(tosses == 1)
tails_count = np.sum(tosses == 0)
print("Heads:", heads_count)
print("Tails:", tails_count)
# (c) Estimate probability of heads
prob_heads = heads_count / 1000
print("Estimated Probability of Heads:", prob_heads)

# Check if it's close to 0.5
if abs(prob_heads - 0.5) < 0.05:
    print("Yes, it's close to 0.5; small random variations may occur.")
```

```
else:
    print("Not very close to 0.5. Could be due to randomness in simulation.")
```

Heads: 499

Tails: 501

Estimated Probability of Heads: 0.499

Yes, it's close to 0.5; small random variations may occur.

Q5

```
In [5]: import pandas as pd
# (a) Create DataFrame
data = {
    'ID': [1, 2, 3, 4],
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Salary': [50000, 60000, 55000, 70000]
}
df = pd.DataFrame(data)
# (b) Add Bonus column = 10% of Salary
df['Bonus'] = df['Salary'] * 0.10
# (c) Display employees with salary above average
average_salary = df['Salary'].mean()
above_avg = df[df['Salary'] > average_salary]
print(above_avg)
```

	ID	Name	Salary	Bonus
1	2	Bob	60000	6000.0
3	4	David	70000	7000.0

Q6

```
In [6]: import numpy as np
# (a) Create a 3x3 array with values 1 to 9
A = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]], dtype=float)
# (b) Transpose
A_T = A.T
# Inverse
try:
    A_inv = np.linalg.inv(A)
except np.linalg.LinAlgError:
    A_inv = None
    print("Matrix is singular and cannot be inverted.")
# (c) Verify  $A \times A^{-1} \approx I$ 
if A_inv is not None:
    identity = np.dot(A, A_inv)
    print("A  $\times$  A-1 =\n", identity)
else:
    print("Cannot verify as inverse does not exist.")
print("Transpose of A =\n", A_T)
```

Matrix is singular and cannot be inverted.
Cannot verify as inverse does not exist.
Transpose of A =
[[1. 4. 7.]
[2. 5. 8.]
[3. 6. 9.]]

Q7

```
In [7]: import numpy as np
# (a) Generate random daily temperatures (30 values between 20 and 40)
temps = np.random.uniform(20, 40, 30)
# (b) Hottest and coldest day
hottest = np.max(temps)
coldest = np.min(temps)
print("Hottest day temperature:", hottest)
print("Coldest day temperature:", coldest)
# (c) Compute mean, median, and standard deviation
mean_temp = np.mean(temps)
median_temp = np.median(temps)
std_temp = np.std(temps)
print("Mean temperature:", mean_temp)
print("Median temperature:", median_temp)
print("Standard deviation:", std_temp)
```

Hottest day temperature: 38.94646568998337
Coldest day temperature: 20.337506353697748
Mean temperature: 30.526898793732055
Median temperature: 31.308851059562812
Standard deviation: 6.2184746538662115

Q8

```
In [9]: import pandas as pd
# (a) Create a Pandas Series of marks
marks = pd.Series([35, 42, 58, 27, 63, 49, 38, 71],
                  index=['Student1', 'Student2', 'Student3', 'Student4', 'Student5',
                        'Student6', 'Student7', 'Student8'])
# (b) Replace all marks below 40 with "Fail"
marks_modified = marks.apply(lambda x: "Fail" if x < 40 else x)
# (c) Count how many students passed
passed_count = sum(isinstance(x, (int, float)) and x >= 40 for x in marks_modified)
print("Modified Marks:\n", marks_modified)
print("Number of students passed:", passed_count)
```

Modified Marks:
Student1 Fail
Student2 42
Student3 58
Student4 Fail
Student5 63
Student6 49
Student7 Fail
Student8 71
dtype: object
Number of students passed: 5

Q9

```
In [10]: import numpy as np
# (a) Generate 500 random integers between 1 and 6
dice_rolls = np.random.randint(1, 7, 500)
# (b) Count how many times each face appears
counts = np.bincount(dice_rolls)[1:] # skip index 0
for face, count in enumerate(counts, start=1):
    print(f"Face {face}: {count} times")
# (c) Compute relative frequencies
relative_freq = counts / 500
for face, freq in enumerate(relative_freq, start=1):
    print(f"Face {face}: relative frequency = {freq:.3f}, theoretical = {1/6:.3f}")
```

Face 1: 81 times
Face 2: 79 times
Face 3: 74 times
Face 4: 95 times
Face 5: 76 times
Face 6: 95 times
Face 1: relative frequency = 0.162, theoretical = 0.167
Face 2: relative frequency = 0.158, theoretical = 0.167
Face 3: relative frequency = 0.148, theoretical = 0.167
Face 4: relative frequency = 0.190, theoretical = 0.167
Face 5: relative frequency = 0.152, theoretical = 0.167
Face 6: relative frequency = 0.190, theoretical = 0.167

Q10

```
In [11]: import pandas as pd
# (a) Create DataFrame
data = {
    'Name': ['Pen', 'Notebook', 'Pencil', 'Eraser', 'Marker', 'Scale'],
    'Quantity': [10, 5, 20, 15, 8, 12],
    'Price': [5, 50, 2, 3, 20, 10]
}
df = pd.DataFrame(data)
# (b) Add column "Total = Quantity × Price"
df['Total'] = df['Quantity'] * df['Price']
```

```
# (c) Find product with maximum sales revenue
max_sales_product = df.loc[df['Total'].idxmax()]
print(df)
print("\nProduct with maximum sales revenue:")
print(max_sales_product)
```

	Name	Quantity	Price	Total
0	Pen	10	5	50
1	Notebook	5	50	250
2	Pencil	20	2	40
3	Eraser	15	3	45
4	Marker	8	20	160
5	Scale	12	10	120

Product with maximum sales revenue:

Name	Notebook
Quantity	5
Price	50
Total	250

Name: 1, dtype: object

Q11

```
In [12]: import pandas as pd
import numpy as np
# (a) Create DataFrame with missing values
data = {
    'A': [1, np.nan, 3, 4],
    'B': [5, 6, np.nan, 8],
    'C': [np.nan, 2, 3, np.nan]
}
df = pd.DataFrame(data)
# (b) Fill missing values with column mean
df_filled = df.fillna(df.mean(numeric_only=True))
# (c) Drop rows where more than 1 value is missing
df_cleaned = df.dropna(thresh=2)
print("Filled DataFrame:\n", df_filled)
print("\nAfter dropping rows with >1 missing values:\n", df_cleaned)
```

Filled DataFrame:

	A	B	C
0	1.000000	5.000000	2.5
1	2.666667	6.000000	2.0
2	3.000000	6.333333	3.0
3	4.000000	8.000000	2.5

After dropping rows with >1 missing values:

	A	B	C
0	1.0	5.0	NaN
1	NaN	6.0	2.0
2	3.0	NaN	3.0
3	4.0	8.0	NaN

Q12

```
In [13]: import numpy as np
# (a) Create NumPy array from 1 to 30
arr = np.arange(1, 31)
# (b) Reshape into 5x6 matrix
matrix = arr.reshape(5, 6)
# (c) Extract even numbers and compute their average
even_nums = matrix[matrix % 2 == 0]
avg_even = even_nums.mean()
print("Matrix:\n", matrix)
print("\nEven numbers:", even_nums)
print("Average of even numbers:", avg_even)
```

Matrix:

```
[[ 1  2  3  4  5  6]
 [ 7  8  9 10 11 12]
 [13 14 15 16 17 18]
 [19 20 21 22 23 24]
 [25 26 27 28 29 30]]
```

Even numbers: [2 4 6 8 10 12 14 16 18 20 22 24 26 28 30]

Average of even numbers: 16.0

Q13

```
In [14]: import pandas as pd
# (a) Create DataFrame
data = {
    'Name': ['Amit', 'Bina', 'Chetan', 'Deepa', 'Ravi', 'Sita'],
    'Age': [19, 22, 18, 21, 20, 19],
    'Marks': [75, 55, 68, 82, 60, 91]
}
df = pd.DataFrame(data)
# (b) Students scoring above overall average
avg_marks = df['Marks'].mean()
above_avg = df[df['Marks'] > avg_marks]
# (c) Students younger than 20 and marks > 60
young_high = df[(df['Age'] < 20) & (df['Marks'] > 60)]
print("Students with marks above average:\n", above_avg)
print("\nStudents younger than 20 with marks > 60:\n", young_high[['Name', 'Age']])
```

Students with marks above average:

	Name	Age	Marks
0	Amit	19	75
3	Deepa	21	82
5	Sita	19	91

Students younger than 20 with marks > 60:

	Name	Age	Marks
0	Amit	19	75
2	Chetan	18	68
5	Sita	19	91

Q14

```
In [15]: import pandas as pd
# (a) Create DataFrame
data = {
    'Name': ['Raj', 'Maya', 'Karan', 'Lina', 'Vikram'],
    'Tasks_Completed': [50, 65, 40, 70, 55],
    'Hours_Worked': [25, 30, 25, 35, 27]
}
df = pd.DataFrame(data)
# (b) Add Efficiency column
df['Efficiency'] = df['Tasks_Completed'] / df['Hours_Worked']
# (c) Identify employee with highest efficiency
max_eff = df.loc[df['Efficiency'].idxmax()]
print(df)
print("\nMost Efficient Employee:")
print("Name:", max_eff['Name'])
print("Efficiency:", max_eff['Efficiency'])
```

	Name	Tasks_Completed	Hours_Worked	Efficiency
0	Raj	50	25	2.000000
1	Maya	65	30	2.166667
2	Karan	40	25	1.600000
3	Lina	70	35	2.000000
4	Vikram	55	27	2.037037

Most Efficient Employee:

Name: Maya

Efficiency: 2.1666666666666665

Case Study 1 – Student Performance Analytics

```
In [16]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
# Simulate data for 100 students
np.random.seed(42)
data = {
    'Student_ID': range(1, 101),
```

```

    'Math': np.random.randint(40, 100, 100),
    'Science': np.random.randint(35, 95, 100),
    'English': np.random.randint(30, 90, 100)
}
df = pd.DataFrame(data)
# (a) Average marks per student & top 5 performers
df['Average'] = df[['Math', 'Science', 'English']].mean(axis=1)
top_5 = df.nlargest(5, 'Average')
print("Top 5 Performers:\n", top_5[['Student_ID', 'Math', 'Science', 'English'])
# (b) Subject-wise average & standard deviation
subject_stats = df[['Math', 'Science', 'English']].agg(['mean', 'std']).T
print("\nSubject-wise Average and Standard Deviation:\n", subject_stats)
# Identify subject with highest variation
highest_variation = subject_stats['std'].idxmax()
print(f"\nSubject with highest variation in marks: {highest_variation}")

# (c) Boxplot for each subject
plt.figure(figsize=(8, 5))
df[['Math', 'Science', 'English']].boxplot()
plt.title("Boxplot of Marks by Subject")
plt.ylabel("Marks")
plt.show()

```

Top 5 Performers:

	Student_ID	Math	Science	English	Average
14	15	92	94	73	86.333333
65	66	92	81	78	83.666667
51	52	90	82	78	83.333333
85	86	81	83	84	82.666667
40	41	99	76	71	82.000000

Subject-wise Average and Standard Deviation:

	mean	std
Math	69.58	18.031499
Science	64.07	16.811645
English	61.14	18.602715

Subject with highest variation in marks: English


```
plt.ylabel("Total Sales")
plt.xlabel("Product")
plt.show()
# Identify underperforming products (low total sales)
underperforming = product_sales[product_sales < product_sales.mean()]
print("\nUnderperforming Products:\n", underperforming)
```

Overall Revenue: 1350

Sales by Region:

Region

East 75

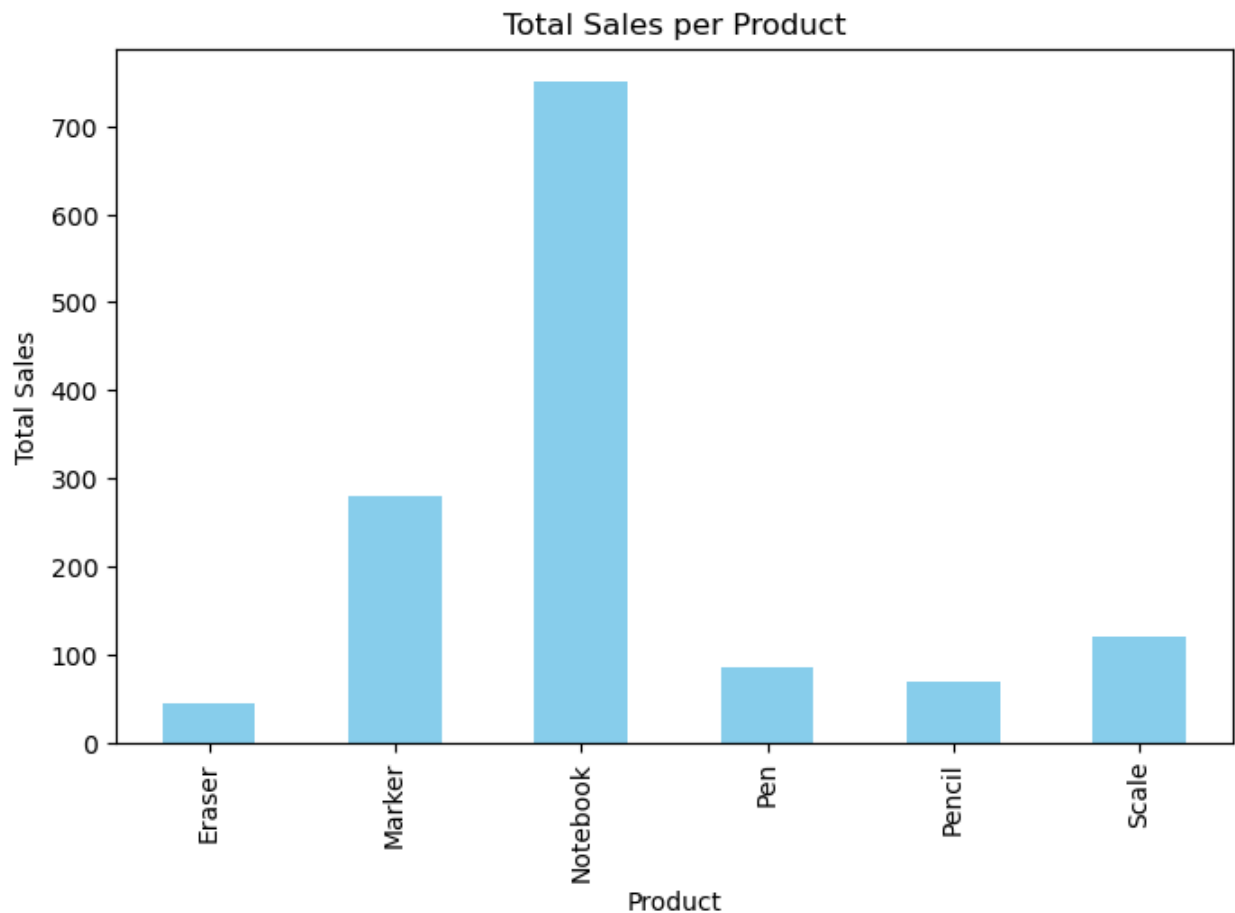
North 240

South 490

West 545

Name: Total, dtype: int64

Region contributing most sales: West



Underperforming Products:

Product

Eraser 45

Pen 85

Pencil 70

Scale 120

Name: Total, dtype: int64

Case Study 3 – Predicting Fuel Efficiency (Auto MPG Dataset)

```
In [20]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
# Load dataset (UCI Auto MPG dataset)
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/auto-mpg/auto-mpg.data"
columns = ['mpg', 'cylinders', 'displacement', 'horsepower', 'weight', 'acceleration', 'model_year', 'origin']
df = pd.read_csv(url, names=columns, sep=' ', na_values='')
# Drop rows with missing values
df = df.dropna()
# Convert horsepower to numeric
df['horsepower'] = df['horsepower'].astype(float)
# Correlation of mpg with numeric features
numeric_features = ['cylinders', 'displacement', 'horsepower', 'weight', 'acceleration', 'model_year', 'origin']
correlations = df[numeric_features + ['mpg']].corr()['mpg'].drop('mpg')
print("Correlation of MPG with numeric features:\n", correlations)
# Scatter plot of weight vs mpg
plt.figure(figsize=(8,5))
sns.scatterplot(x='weight', y='mpg', data=df)
plt.title("Scatter Plot: Vehicle Weight vs MPG")
plt.xlabel("Weight")
plt.ylabel("MPG")
plt.show()
```

Correlation of MPG with numeric features:

cylinders	-0.777618
displacement	-0.805127
horsepower	-0.778427
weight	-0.832244
acceleration	0.423329
model_year	0.580541
origin	0.565209

Name: mpg, dtype: float64

Scatter Plot: Vehicle Weight vs MPG

